

BankLite — Cross Platform Banking Application (Documentation)

Name: Sweta P Kadam

Roll No.: 150096724176

Subject: Cross Platform Application Laboratory (Semester V)

Project Title: 79 BankLite — Cross Platform Banking Application

[Live Demo: BankLite](#)

[GitHub Repository: BankLite](#)

[Figma Design: View Figma Design](#)

1. Introduction

Modern banking apps have become cluttered and difficult to navigate. Users often struggle to find basic features like sending money or paying bills behind layers of promotional menus and unnecessary screens.

BankLite was built to solve this problem by offering a clean, distraction-free digital banking interface. It provides everything a user needs — checking their balance, sending money, paying utility bills, tracking spending, and downloading certified statements — all from a single, well-organized app.

1.1 Problem Statement

Students, working professionals, and seniors often find banking apps overwhelming due to:

- Too many promotional popups and unrelated content
- Deep nested navigation to reach simple features
- No clear spending visibility or budget tracking
- No easy way to generate or download account statements

BankLite addresses these issues with a minimal, task-focused interface backed by Firebase for real-time cloud data storage.

1.2 Objectives

- Build a fully functional cross-platform banking app using Flutter
- Integrate Firebase Authentication for secure user login and registration

- Store all user data (accounts, transactions, bills, statements) in Cloud Firestore
- Provide certified PDF and Excel account statements generated on-device
- Deploy the app as a live Progressive Web App (PWA) on Firebase Hosting

2. App Overview

BankLite is a cross-platform mobile banking application developed using Flutter and Firebase. It simulates a real digital banking experience with features including fund transfers, utility bill payments, budget insights, transaction history, and certified account statements.

2.1 App Summary

Parameter	Details
App Name	BankLite
Platform	iOS, Android, Web, macOS
Framework	Flutter 3.29 (Dart 3.7)
Backend	Firebase Auth + Cloud Firestore
Hosting	Firebase Hosting (PWA)
Live Link	https://banklite-f95a6.web.app
GitHub	https://github.com/Shweta-Tech-creator/banklite
Figma Prototype	Interactive design prototype
Total Screens	9 (Splash, Login, Register, Dashboard, Transfer, Bills, Transactions, Budget, Profile)
Total Features	7 core modules

2.2 Key Capabilities

The app provides the following major capabilities:

- **Account Management:** Dual-account support (Savings + Current) with real-time balance display
- **Fund Transfer:** Send money to saved beneficiaries with instant balance deduction
- **Bill Payments:** Pay Electricity, Mobile Recharge, Broadband, Water, and Fastag bills
- **Transaction History:** Searchable, filterable ledger with digital receipts
- **Budget Insights:** Category-wise spending breakdown with visual progress bars

- **Account Statements:** On-device PDF and Excel (CSV) statement generation
- **Cloud Sync:** All data stored per-user in Firebase Firestore with offline fallback

3. Tech Stack & Architecture

Before building the features, the overall app architecture was designed to ensure clean separation of concerns, scalability, and offline resilience.

3.1 Technology Stack

Layer	Technology	Purpose
UI Framework	Flutter 3.29 / Dart 3.7	Cross-platform UI rendering
State Management	ChangeNotifier + InheritedWidget	Reactive UI updates
Authentication	Firebase Auth 6.7	Secure user login and registration
Database	Cloud Firestore 6.10	Real-time cloud data storage
Local Cache	SharedPreferences 2.5	Offline data persistence
PDF Export	pdf 3.13	Certified statement generation
Hosting	Firebase Hosting	Live PWA deployment
Version Control	Git + GitHub	Source control

3.2 Project Structure

The app is organized into the following key folders:

```

banklite/
├── lib/
│   ├── models/      → Data models (Account, Transaction, Bill, Budget, User)
│   ├── screens/     → All app screens (Login, Dashboard, Transfer, Bills, etc.)
│   ├── services/    → Firebase service layer and state management
│   ├── widgets/     → Reusable UI components (BalanceCard, Receipts, Statements)
│   ├── utils/       → PDF generator, currency formatter, file downloader
│   └── main.dart     → App bootstrap and routing
├── firestore.rules   → Cloud Firestore security rules
└── pubspec.yaml      → Dependencies
  
```

3.3 Key Architecture Decisions

- 1. **Resilient Data Layer:** Works with live Firestore when connected and automatically falls back to local SharedPreferences cache when offline, ensuring the app never crashes.
- 2. **Optimistic UI Updates:** Balance deductions and transaction entries are applied locally first, then synced to Firestore in the background, giving users an instant response.
- 3. **Client-Side Exports:** PDF and Excel statements are generated entirely in Dart on the device without any server dependency, ensuring privacy and instant downloads.

4. Firebase Data Architecture

Since BankLite is a real banking simulation, all user data is stored securely in Cloud Firestore. Each user's data is completely isolated under their own document path.

4.1 Firestore Data Structure

All data for each user is stored under the path `/users/{userId}/:`

Collection Path	Purpose	Key Fields
<code>/users/{uid}</code>	User profile document	name, email, phone, accountNumber, customerTier, isKycVerified
<code>/users/{uid}/accounts/{accountId}</code>	Bank accounts & balances	accountType, balance, accountNumber, maskedNumber
<code>/users/{uid}/transactions/{transactionId}</code>	Full transaction ledger	title, amount, type, category, date, status, recipientOrSender
<code>/users/{uid}/bills/{billId}</code>	Utility bill payments	operator, category, amount, consumerNumber, paymentId
<code>/users/{uid}/beneficiaries/{benId}</code>	Saved payees	name, bankName, accountNumber, maskedNumber
<code>/users/{uid}/budget/current</code>	Monthly budget	monthlyBudget, totalSpent, categorySpending
<code>/users/{uid}/statements/{statementId}</code>	Statement records	format, period, generatedAt, accountNumber

4.2 Security Rules

Firestore security rules ensure that each user can only access their own data. No user can read or write another user's records.

```
match /users/{userId} {  
  allow read, write: if request.auth != null  
    && request.auth.uid == userId;  
  match /{allSubcollections=**} {  
    allow read, write: if request.auth != null  
      && request.auth.uid == userId;  
  }  
}
```

4.3 Data Sync Strategy

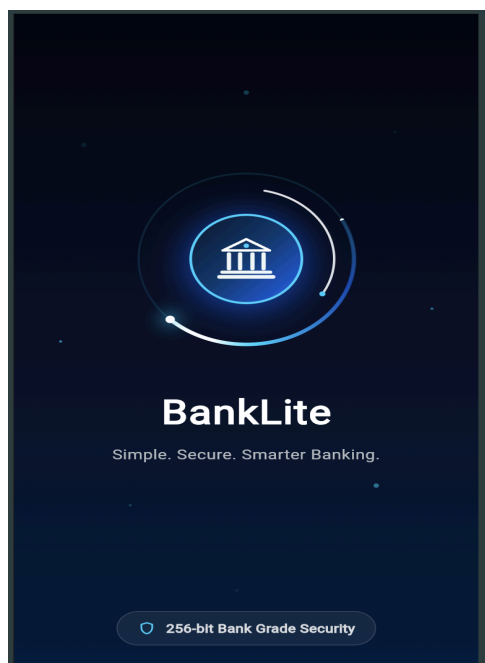
The app uses a dual-layer persistence strategy:

- **Primary:** Real-time Firestore streams that update the UI live as data changes
- **Fallback:** SharedPreferences local cache loaded instantly on app launch
- **Write Guard:** A 15-second guard prevents stale Firestore snapshots from overwriting optimistic local balance updates after a transfer

5. Core Features Implemented

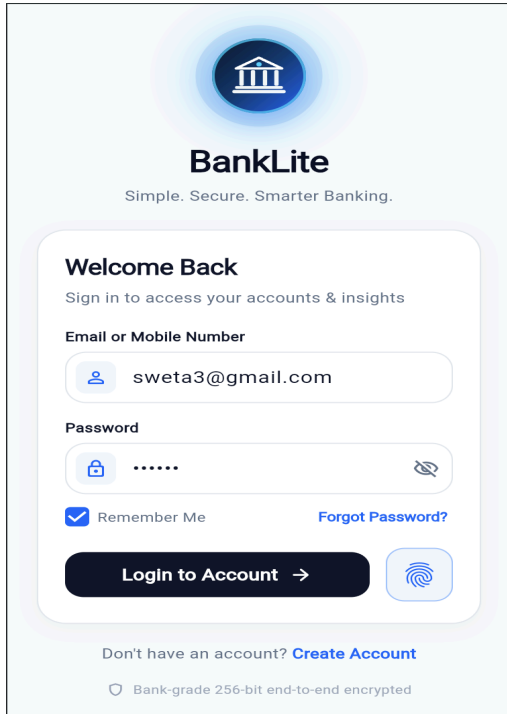
For this project, seven core banking modules were designed and implemented.

Feature 1: Splash Screen & Onboarding



An animated splash screen displays the BankLite emblem with a smooth scale and fade animation before routing the user to the login screen. The app checks for an existing persistent session and skips login if the user is already authenticated.

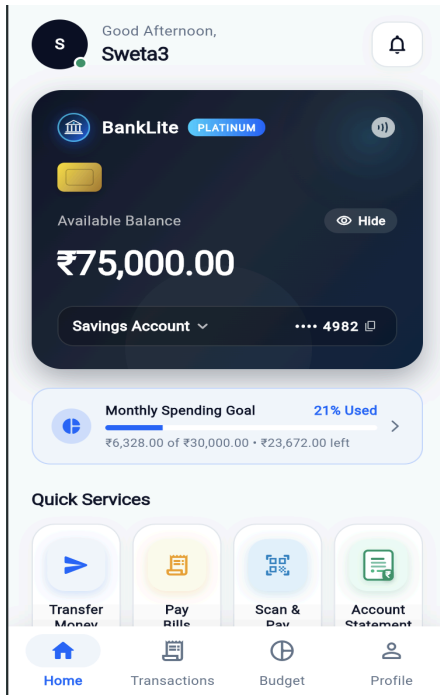
Feature 2: Login & Registration with Firebase Auth



The image shows a mobile app login screen for 'BankLite'. At the top, there is a blue circular logo with a white bank building icon. Below the logo, the text 'BankLite' is displayed in a bold, dark font, followed by the tagline 'Simple. Secure. Smarter Banking.' in a smaller, lighter font. The main content area is a white rounded rectangle with a light blue border. It starts with the heading 'Welcome Back' and a subtext 'Sign in to access your accounts & insights'. Below this, there are two input fields: 'Email or Mobile Number' with the value 'sweta3@gmail.com' and a 'Password' field with masked characters '.....'. To the right of the password field is an eye icon for toggling visibility. Below the password field, there is a 'Remember Me' checkbox (checked) and a 'Forgot Password?' link. At the bottom of the login area, there is a dark blue button labeled 'Login to Account' with a right arrow, and a circular fingerprint icon to its right. At the very bottom of the screen, there is a link 'Don't have an account? Create Account' and a security note 'Bank-grade 256-bit end-to-end encrypted' with a lock icon.

Users can log in using email and password via Firebase Authentication. The app also simulates biometric fingerprint authentication. New users are registered with Firebase Auth, and their profile, two bank accounts, an opening deposit transaction, and default beneficiaries are immediately written to Firestore as a batch operation.

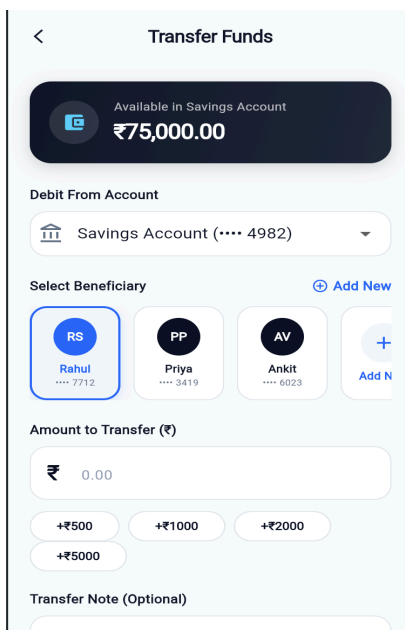
Feature 3: Dashboard with Balance Card



The home dashboard displays:

- A luxury account card showing the active account type, masked number, and balance
- A privacy toggle to hide or reveal the balance
- A real-time monthly spending progress bar
- Four quick-action cards: Transfer Money, Pay Bills, Scan & Pay, Account Statement

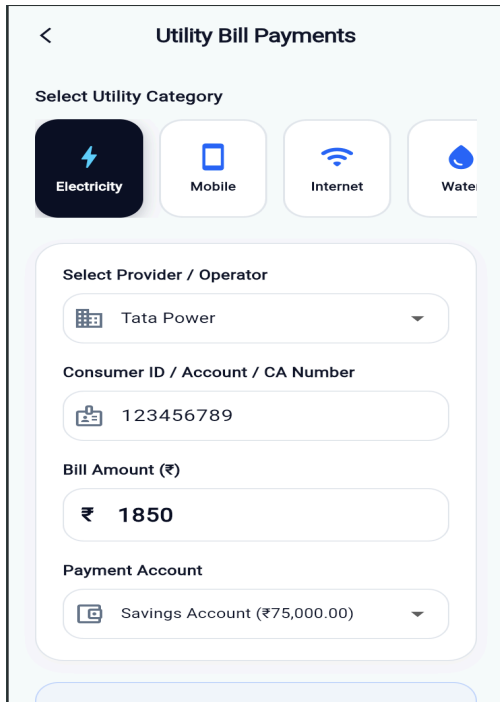
Feature 4: Fund Transfer



The transfer screen allows users to:

- Select a saved beneficiary from a horizontal carousel
- Enter a transfer amount with quick-fill chips (₹500, ₹1,000, ₹2,000, ₹5,000)
- Review a confirmation sheet showing recipient, amount, and charges (₹0)
- Confirm the transfer, which instantly deducts the balance locally and commits to Firestore

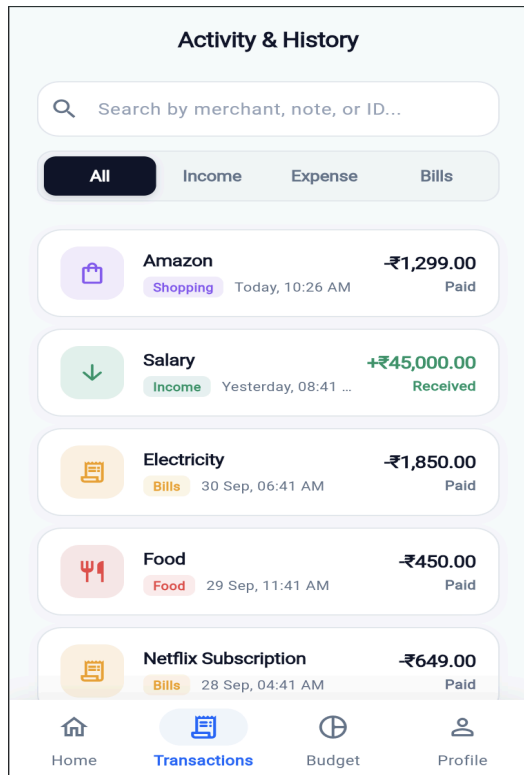
Feature 5: Utility Bill Payments



The screenshot shows a mobile app interface for "Utility Bill Payments". At the top, there is a back arrow and the title "Utility Bill Payments". Below the title, the instruction "Select Utility Category" is followed by four buttons: "Electricity" (selected, with a lightning bolt icon), "Mobile" (with a mobile phone icon), "Internet" (with a Wi-Fi icon), and "Water" (with a water drop icon). Below these buttons, the instruction "Select Provider / Operator" is followed by a dropdown menu showing "Tata Power". Below that, the instruction "Consumer ID / Account / CA Number" is followed by a text input field containing "123456789". Below that, the instruction "Bill Amount (₹)" is followed by a text input field containing "₹ 1850". Finally, the instruction "Payment Account" is followed by a dropdown menu showing "Savings Account (₹75,000.00)".

Supported bill categories: Electricity (Tata Power, Adani, BESCOM), Mobile Recharge (Jio, Airtel, Vi), Broadband Internet, Water, and Fastag. Each payment includes consumer ID validation, convenience fee display, and automatic ledger logging.

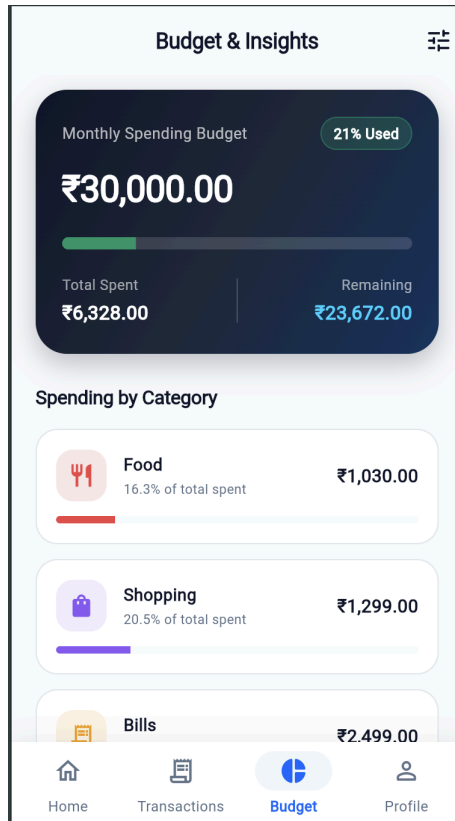
Feature 6: Transaction History & Search



A searchable and filterable transaction ledger with:

- Filter chips: All, Income, Expense, Bills
- Live search by payee name, category, note, or transaction reference ID
- Tap-to-open digital receipt dialog with status badges

Feature 7: Budget Insights



A visual budget dashboard showing:

- Monthly spending limit vs. total spent
- Category-wise percentage bars for Food & Dining, Shopping, Utilities, and Travel
- Proactive financial health tips when spending approaches the limit

6.2 Design Principles

- **Mobile-First:** Designed for phone screens with a responsive wrapper that centers the layout on tablets, web browsers, and desktop
- **Clean Color System:** Navy blue (#1a237e) primary, white cards, green for credits, red for debits
- **Luxury Card Style:** Rounded corners, gradient balance card, masked account number
- **Consistent Typography:** Bold headings, regular body text, color-coded amounts

8. Firebase Data Verification

8.1 Why Firebase Firestore Was Selected

Cloud Firestore was selected as the backend because:

- It provides **real-time listeners** so the UI updates automatically when data changes
- It supports **offline persistence** so the app works without internet
- It scales automatically without managing servers
- Security rules allow **strict per-user data isolation**
- It integrates natively with Firebase Auth for zero-config authentication

8.2 Confirmed Data in Firebase Console

After running the verification script, the following data was confirmed visible in the Firebase Console under `/users/gHQWwoVLMuewdZp6XKTjYok8kAs1/`:

Collection	Documents Found	Sample Data
accounts	1	Savings Account, ₹75,000 balance
transactions	1	"Verification Transfer Test", ₹500, debit
bills	1	"BEST Electricity", ₹1,200
beneficiaries	1	"Test Beneficiary", HDFC Bank
budget	1	Monthly budget ₹30,000, spent ₹1,700
statements	1	PDF, Sep 2026

8.3 Real User Data (sweta3@gmail.com)

The real user account for `sweta3@gmail.com` is visible in Firebase Console under the `users` collection with:

- Profile fields: name, email, phone, accountNumber, customerTier, isKycVerified
- Subcollection: `accounts` with Savings and Current accounts
- Additional subcollections (bills, transactions, statements) are created automatically when the user performs those actions in the deployed app

9. Live Web App (Firebase Hosting)

To make the project accessible without installing anything, the app was compiled and deployed as a Progressive Web App (PWA) using Firebase Hosting.

Deployment Steps

1. **Build web release:**
bash
2. flutter build web --release
3. **Configure Firebase Hosting** (`firebase.json` pointing to `build/web`)
4. **Deploy:**
bash
5. firebase deploy --only hosting,firestore:rules

Features of the Live App

- **Instant Access:** Works in any browser — Chrome, Safari, Firefox — without installing
- **PWA Support:** Can be added to the home screen on Android and iOS like a native app
- **Firebase Auth:** Full login and registration works in the browser
- **Real-Time Sync:** Transfers, bill payments, and budget updates sync to Firestore live
- **Responsive Layout:** Mobile banking card is centered on desktop screens for authentic feel
- **Offline Fallback:** SharedPreferences cache ensures the app loads even without internet

Live URL: <https://banklite-f95a6.web.app>

GitHub Repository: <https://github.com/Shweta-Tech-creator/banklite>

10. Conclusion & Future Scope

10.1 Conclusion

This project demonstrates a fully functional cross-platform banking application built with Flutter and Firebase. The app provides a clean, distraction-free interface for fund transfers, utility bill payments, transaction tracking, budget insights, and certified account statements. All user data is stored securely in Cloud Firestore with per-user isolation enforced by Firestore security rules. The app passed all 13 automated unit tests and all 10 Firebase verification checks. It is deployed as a live Progressive Web App on Firebase Hosting.

10.2 Future Scope

- **UPI Integration:** Add real UPI payment processing via Razorpay or Stripe APIs
- **Loan & Credit Module:** Display loan EMI schedules and credit score summaries
- **Spending Forecasting:** Predict next month's spending based on transaction history using ML
- **Biometric Auth:** Full Face ID and Fingerprint authentication using `local_auth` plugin
- **Push Notifications:** Real-time Firebase Cloud Messaging alerts for every debit and credit
- **Multi-Currency Support:** Add foreign currency accounts for international transfers
- **Dark Mode:** Full dark theme toggle across all screens

- **Admin Dashboard:** Web panel for viewing all registered users and transaction volumes